

Agentic Recommender System with Hierarchical Belief-State Memory

Xiang Shen*, Yuhang Zhou*, Yifan Wu*, Zhuokai Zhao, Siyu Lin, Lei Huang, Qianqian Zhong, Lizhu Zhang, Benyu Zhang, Xiangjun Fan, Hong Yan

Meta Recommendation Systems (MRS)

*Equal contribution

Memory-augmented LLM agents have advanced personalized recommendation, yet existing approaches universally adopt flat memory representations that conflate ephemeral signals with stable preferences, and none provides a complete lifecycle governing how memory should evolve. We propose MARS (Memory-Augmented Agentic Recommender System), a framework that treats recommendation as a partially observable problem and maintains a structured belief state that progressively abstracts noisy behavioral observations into a compact estimate of user preferences. MARS organizes this belief state into three tiers: event memory buffers raw signals, preference memory maintains fine-grained mutable chunks with explicit strength and evidence tracking, and profile memory distills all preferences into a coherent natural language narrative. A complete lifecycle of six operations—extraction, reinforcement, weakening, consolidation, forgetting, and resynthesis—is adaptively scheduled by an LLM-based planner rather than fixed-interval heuristics. Experiments on four InstructRec benchmark domains show that MARS achieves state-of-the-art performance with average improvements of 26.4% in HR@1 and 10.3% in NDCG@10 over the strongest baselines with further gains from agentic scheduling in evolving settings.

Correspondence: xiangshen@meta.com, maxfan@meta.com

Date: May 18, 2026



1 Introduction

Recommender systems are fundamental to online content discovery, powering personalization across social media, e-commerce, and entertainment platforms at billion-user scale. The field has progressed from collaborative filtering (Rendle et al., 2009) and graph-based methods (He et al., 2020) to deep learning ranking models (Naumov et al., 2019), and more recently to self-attentive sequential architectures (Kang and McAuley, 2018; Zhai et al., 2024) that capture temporal dynamics of user behavior. While these systems achieve strong offline metrics, they share two fundamental limitations. First, users are passively involved, able to influence recommendations only through implicit signals or blunt controls rather than using natural language to directly express intent and steer the system (Carroll et al., 2025; Zhang et al., 2023). Second, embedding-based models act as narrow experts that capture statistical co-occurrences but cannot reason about *why* an item matches a user’s taste (Zhao et al., 2024; Peng et al., 2025).

The emergence of large language models (LLMs) has opened a new paradigm that addresses these limitations by enabling systems to reason about user preferences and item semantics in natural language. While LLMs can rank items zero-shot (Hou et al., 2024; Lyu et al., 2024), without persistent memory they cannot accumulate understanding across interactions, motivating a growing body of memory-augmented agentic recommender systems (Peng et al., 2025). These systems range from static methods that construct or rebuild user profiles from interaction histories (Xu et al., 2025; Shi et al., 2025), to dynamic approaches that evolve preferences incrementally through reflection and collaborative signal propagation (Zhang et al., 2024b; Liao et al., 2026; Chen et al., 2026; Nguyen et al., 2026; Li et al., 2026). Despite this progress, all existing approaches share critical limitations in how memory is structured and managed over its lifetime.

These limitations manifest in two fundamental gaps. First, all existing systems adopt flat memory representations that fail to effectively perform state abstraction over the recommendation environment. Specifically, they

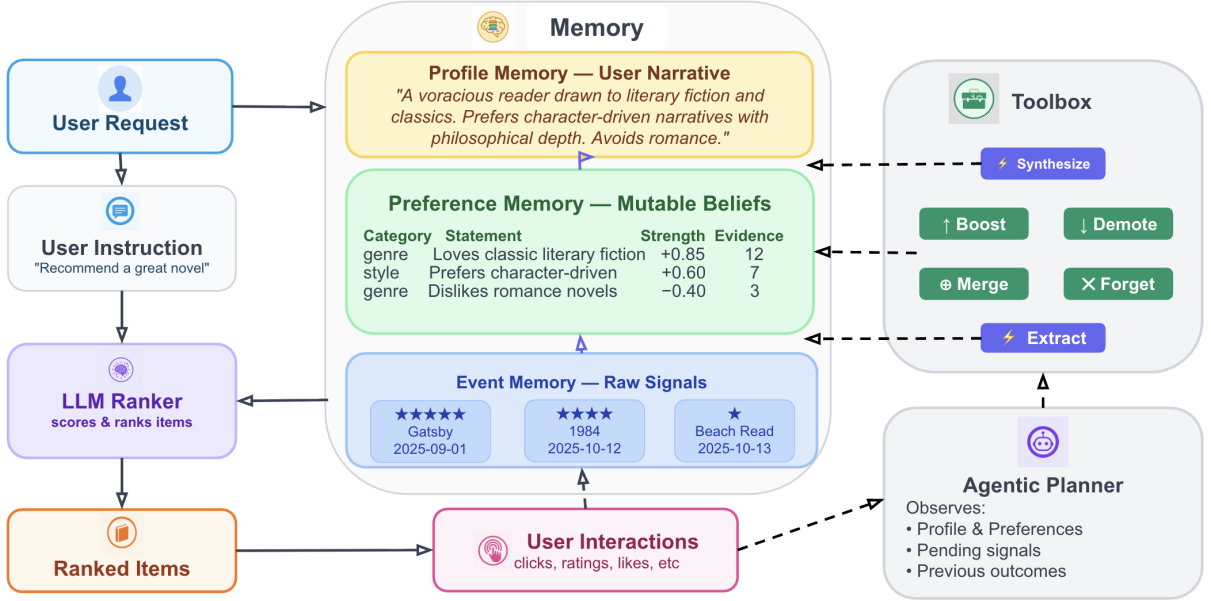


Figure 1 Overview of MARS. Three-tier memory (center) stores raw signals (event), mutable preference chunks with strength and evidence (preference), and a synthesized user narrative (profile). Solid arrows: online ranking path where profile and event memory feed the LLM ranker. Dashed arrows: offline lifecycle path where an agentic planner schedules six memory operations to keep preferences current. Preference memory serves as an intermediate tier that informs profile synthesis but is not directly consumed by the ranker. The two paths are decoupled.

conflate three crucial abstraction levels: raw, high-dimensional observations (the *momentary* level—what the user is doing now), disentangled belief variables (the *dispositional* level—what they consistently prefer, and with what confidence), and the holistic state representation (the *identity* level—who they are as a consumer). By collapsing these distinct tiers, a flat representation loses either the granularity for surgical credit assignment, the recency for detecting distribution shifts, or the semantic coherence required for accurate ranking. Second, no existing system provides a formal state transition mechanism—a complete memory lifecycle—governing when and how the belief state should evolve, leaving no principled method for extracting, consolidating, or decaying preferences as new evidence accumulates.

To address these gaps, we propose **MARS** (Memory-Augmented Agentic Recommender System), an agentic recommendation framework that formulates memory management through the lens of a Partially Observable Markov Decision Process (Kaelbling et al., 1998; Lu and Yang, 2016). To resolve the partial observability of underlying user intents, MARS implements a hierarchical state abstraction architecture, assigning each level of abstraction its own dedicated tier: *event memory* buffers raw interaction signals (the observation space), *preference memory* maintains fine-grained, independently mutable preference chunks with explicit strength and evidence tracking (the disentangled belief variables), and *profile memory* distills these discrete chunks into a coherent natural language narrative (the holistic state representation). Each tier specializes for a distinct role: events capture raw transition dynamics, preferences enable surgical credit assignment where new evidence updates only the relevant belief without disturbing unrelated variables in the state space, and the profile provides the compact, Markovian context that the LLM-based ranker consumes most effectively.

In addition, MARS introduces the first complete memory lifecycle for agentic recommendation, governing how information transitions across these tiers through a principled set of operations: extraction of preferences from raw signals, reinforcement and weakening as new evidence accumulates, consolidation of near-duplicate beliefs, evidence-driven forgetting, and profile resynthesis. Unlike prior systems that either update memory after every interaction or rebuild it at fixed intervals, an LLM planner adaptively schedules these operations based on accumulated signal volume, making context-aware decisions about when memory maintenance is warranted and thereby balancing memory freshness against computational cost.

Our primary contributions can be summarized as follows:

- We propose **MARS**, an agentic recommender system built around a **three-tier memory architecture** (event, preference, and profile memory) that resolves the ambiguity between short-term behavioral signals and long-term user traits by progressively abstracting raw events into mutable preference chunks and then into coherent natural language profiles.
- We design a **complete memory lifecycle** encompassing extraction, reinforcement, weakening, consolidation, forgetting, and resynthesis, governed by an LLM-based planner that adaptively schedules operations based on accumulated context, replacing rigid heuristics and reducing computational cost.
- We conduct **extensive experiments** on the InstructRec benchmark (Xu et al., 2025) across four domains in both retrospective and evolving evaluation settings, demonstrating state-of-the-art performance with average improvements of 26.4% in HR@1 and 10.3% in NDCG@10 over the strongest baseline per domain.

2 Related Work

2.1 LLM Agents in Recommendation Systems

The application of LLM-powered agents to recommendation systems has attracted considerable attention (Peng et al., 2025). One line of work employs LLM agents to simulate user behavior for evaluation and analysis (Zhang et al., 2024a; Wang et al., 2023; Zhong et al., 2025; Shi et al., 2025; Bougie and Watanabe, 2025; Chen et al., 2025), demonstrating that LLM agents can accurately reproduce user behavior but contributing primarily to evaluation methodology rather than recommendation quality. A separate line of work leverages LLMs as autonomous agents that orchestrate conventional recommender models as tools (Wang et al., 2024; Huang et al., 2023; Lei et al., 2024; Zhao et al., 2024; Deng et al., 2025; Ou et al., 2026). While these systems exhibit strong reasoning and planning capabilities, they operate without persistent user memory, treating each session independently.

The most closely related work equips LLM agents with persistent memory to model evolving user preferences. iAgent (Xu et al., 2025) introduced a user-agent-platform paradigm that mitigates platform-side biases on behalf of the user, with i2Agent extending it through iterative profile generation via feedback loops. AgentCF (Zhang et al., 2024b) modeled both users and items as autonomous agents that collaboratively update their memories through forward interaction and backward reflection. STEAM (Liao et al., 2026) introduced atomic memory units that form communities and prototypes, incorporating memory formation and consolidation but lacking a forgetting mechanism. Since these per-user approaches cannot leverage collaborative signals, more recent work propagates preference updates across users through global memory pools (Nguyen et al., 2026), router-mediated networks (Li et al., 2026), or architecturally decoupled memory managers with collaborative graph propagation (Chen et al., 2026). Despite this progress, all existing approaches adopt flat memory representations and lack a complete lifecycle governing extraction, consolidation, and forgetting. MARS addresses these gaps through a three-tier architecture governed by a full lifecycle with adaptive scheduling.

2.2 LLM Agent Memory

Memory management has emerged as a central challenge in LLM agent systems. A dominant design paradigm draws inspiration from operating systems: MemGPT (Packer et al., 2023) pioneered virtual context management with self-directed paging between main context and external storage, and subsequent work extended this with universal memory encapsulation and cross-type migration (Li et al., 2025) or heat-based tier promotion (Kang et al., 2025). Rather than OS-level abstractions, an alternative line of work represents memory as structured knowledge through incremental CRUD stores (Chhikara et al., 2025) or temporal knowledge graphs (Rasmussen et al., 2025).

Alongside architectural advances, memory evolution mechanisms have become an active area of research. CoALA (Sumers et al., 2023) mapped cognitive science memory types onto language agents and explicitly identified memory modification and deletion as “understudied,” while MemoryBank (Zhong et al., 2024) and H-Mem (Ye et al., 2026) addressed forgetting through Ebbinghaus decay curves and feedback-driven weight

adjustment, respectively. Despite these advances, no existing system combines extraction, consolidation, and forgetting into a unified lifecycle; most handle forgetting through simple heuristics such as FIFO eviction or time-based decay. Moreover, these systems are designed for factual recall and conversational coherence rather than recommendation, where memory must additionally capture preference intensity, evidence strength, and multi-timescale dynamics. MARS bridges this gap by adapting hierarchical memory principles to preference modeling with domain-specific lifecycle management.

3 Methodology

3.1 Problem Formulation

We consider the sequential recommendation problem with natural language instructions. Let $\mathcal{U}$ denote the set of users and $\mathcal{I}$ denote the set of items. Given a user $u \in \mathcal{U}$ with historical interactions $\mathcal{H}_u = \{(i_1, t_1), (i_2, t_2), \dots, (i_T, t_T)\}$ ordered by timestamp where each $i_k \in \mathcal{I}$, an optional natural language instruction q_u expressing the user’s current intent, and a candidate set $\mathcal{C} \subseteq \mathcal{I}$ of items to rank, the goal is to produce a ranking of $\mathcal{C}$ that maximizes relevance to the user’s preferences and intent.

To model the uncertainty of user intent, we draw on the POMDP framework (Kaelbling et al., 1998; Lu and Yang, 2016). The user’s true preferences constitute a hidden state $s \in \mathcal{S}_{\text{hidden}}$ governed by unknown transition dynamics $\mathcal{T}(s' | s, a)$; the system cannot observe s directly but receives partial, noisy observations $o \in \Omega$ through behavioral signals (clicks, purchases, ratings). In a standard POMDP, an agent maintains a probabilistic belief state updated via Bayesian filtering, but this is intractable when preferences live in open-ended natural language. MARS instead maintains a structured, symbolic belief state $M_u = (\mathcal{E}_u, \mathcal{P}_u, \mathcal{S}_u)$ that progressively abstracts raw observations ($\mathcal{E}_u$) into discrete, disentangled belief variables ($\mathcal{P}_u$) and a holistic semantic representation ($\mathcal{S}_u$). An LLM-based planner $\pi_{\text{plan}}(a_t | M_u)$ selects discrete memory-updating actions $a_t \in \mathcal{A}$ (e.g., extract, boost, demote, merge, forget, synthesize), balancing memory freshness against computational cost, while a ranking policy π_{rank} generates the item list from $\mathcal{C}$ conditioned on M_u and the user instruction q_u .

3.2 Three-Tier Memory Architecture

Following our POMDP formulation, MARS constructs its symbolic belief state M_u through three tiers of increasing abstraction, as illustrated in Figure 1.

Event Memory. The first tier $\mathcal{E}_u = \{e_1, e_2, \dots, e_L\}$ captures the momentary level, storing raw behavioral signals as structured records. Each signal $e = (u, i, a, m_i, t)$ captures the user u , item i , action type a , item metadata m_i , and timestamp t . In our framework, this tier serves as the raw observation space Ω , faithfully recording behavioral data ($o \in \Omega$) without interpretation. Event memory is maintained as a bounded FIFO queue with capacity L_{max} . We denote by $\mathcal{E}_u^{\text{pending}} \subseteq \mathcal{E}_u$ the subset of signals that have not yet been processed by preference extraction; once processed, observations remain in $\mathcal{E}_u$ as ranking context but are no longer pending.

Preference Memory. The second tier $\mathcal{P}_u = \{p_1, p_2, \dots, p_K\}$ captures the dispositional level, representing the user’s preferences as fine-grained, independently mutable natural language statements acting as disentangled belief variables. Each preference chunk p is a tuple of four fields: a domain-specific category label (e.g., genre, writing style), a natural language statement describing the preference, a strength $\in [-1, 1]$ score indicating intensity and polarity, and an evidence $\in \mathbb{N}$ count tracking how many interactions have reinforced or contradicted this belief (see Figure 1 for an example). These strength and evidence fields act as explicit confidence intervals over the belief state. Preference chunks are constructed by an LLM-based extractor π_{ext} that acts as an observation-to-belief mapping, analyzing pending event signals against existing preferences to produce structured updates:

$$\Delta \mathcal{P} = \pi_{\text{ext}}(\mathcal{E}_u^{\text{pending}}, \mathcal{P}_u)$$

Because each chunk is independently mutable, new evidence updates only the relevant belief dimensions rather than globally rewriting the user representation, enabling precise credit assignment. Preference memory feeds directly into profile memory, where the full set of discrete variables is consolidated into a cohesive narrative.

Profile Memory. The third tier $\mathcal{S}_u$ captures the identity level, distilling all preference chunks into a coherent natural language narrative of the user’s overall taste profile. The profile is generated by an LLM-based synthesizer π_{syn} :

$$\mathcal{S}_u = \pi_{\text{syn}}(\mathcal{P}_u, \mathcal{S}_u^{\text{prev}})$$

where $\mathcal{S}_u^{\text{prev}}$ is the previous profile version, included to maintain narrative continuity across state updates. Beyond aggregating individual preference chunks, the synthesized profile captures relationships between preferences and resolves potential tensions, producing the most compressed, Markovian state representation that the ranker consumes alongside recent behavioral signals.

This three-tier design offers key advantages over flat representations. Profile resynthesis is amortized over multiple mutations rather than triggered after every interaction, yielding computational savings. Both preference and profile tiers are expressed in natural language, making the belief state fully interpretable and providing a semantically rich input for LLM-based ranking. However, without proactive pruning and consolidation, the preference tier will inevitably accumulate redundant or contradicted beliefs, introducing high-dimensional noise. This motivates a formal transition mechanism to govern state maintenance.

3.3 Memory Lifecycle Management

The memory lifecycle governs the transition dynamics $\mathcal{T}(s' | s, a)$ of the belief state M_u as new observations arrive. It defines six primitive operations that constitute the discrete action space $\mathcal{A}$ exposed to an LLM-based planner π_{plan} , which acts as a transition policy selecting actions based on the current belief state.

Memory Operations. Five operations act on preference memory $\mathcal{P}_u$ and one on profile memory $\mathcal{S}_u$. The preference operations serve as deterministic, heuristic alternatives to exact Bayesian belief updates:

- **Extract:** invokes π_{ext} to create new preference chunks from pending signals. In evolving mode, this is the only tool requiring an LLM call; the remaining operations are deterministic state updates.
- **Boost:** reinforces a chunk by incrementing its strength and evidence count, increasing state confidence.
- **Demote:** weakens a chunk by decrementing strength. The demote step δ_- is larger than the boost step δ_+ , implementing a pessimistic update rule where contradictory evidence erodes beliefs faster than confirmatory evidence reinforces them.
- **Merge:** consolidates near-duplicate chunks identified by the planner, directly reducing state collinearity and redundancy.
- **Forget:** prunes chunks whose strength has decayed below a threshold after sufficient evidence, acting as an explicit decay function that removes stale or contradicted state variables.
- **Synthesize:** regenerates $\mathcal{S}_u$ from $\mathcal{P}_u$ via π_{syn} . Also triggered automatically every γ mutations as a safeguard.

After any mutation sequence, a per-category capacity constraint retains only the top K_{cat} chunks ranked by $\text{evidence}_p \cdot |\text{strength}_p|$, mathematically bounding the dimensionality of the belief state.

Agentic Scheduling. Rather than triggering operations on a fixed schedule, the planner π_{plan} acts as a frozen heuristic policy over the belief state, selecting which actions $a_t \in \mathcal{A}$ to apply:

$$a_t = \pi_{\text{plan}}(\mathcal{S}_u, \mathcal{P}_u, \mathcal{E}_u^{\text{pending}}, H_{\text{prev}})$$

where H_{prev} records outcomes of the previous round. The planner receives the current belief state—profile, preference chunks with confidence metadata, pending observations, and a memory health summary—and outputs either an empty action list (skip, the default when new observations are strictly consistent with existing beliefs) or an ordered sequence of transitions $a_t = [(t_1, \text{params}_1), \dots, (t_m, \text{params}_m)]$ over the plannable operations. The skip action reflects an implicit reward optimization: when new signals carry no marginal information, the expected advantage of a state transition is negative, meaning the computational cost outweighs the utility. In practice, the policy invokes extraction when observations suggest novel latent intents, schedules merge when state redundancy is detected, and triggers forgetting when belief strength decays below optimal thresholds. The full procedure is detailed in Algorithm 1 (Appendix A).

3.4 LLM-based Ranking

Given the abstracted belief state M_u and candidate set $\mathcal{C}$, MARS produces a ranked list through pointwise scoring. The ranking prompt incorporates two forms of user context: the holistic state representation $\mathcal{S}_u$ and recent behavioral observations from $\mathcal{E}_u$. Preference memory $\mathcal{P}_u$ is not directly included in the ranking prompt; instead, its contribution is mediated through profile synthesis, which distills the discrete variables into the coherent narrative consumed by the ranker. This design avoids redundancy between raw preference chunks and the profile that already summarizes them, as validated by the ablation study in §4.4. When a natural language instruction q_u is provided, it receives explicit priority over historical beliefs. All candidates are presented in a single prompt, and the ranking policy assigns each an independent relevance score along with a brief rationale:

$$\{r_1, \dots, r_N\} = \pi_{\text{rank}}(\mathcal{C}, \mathcal{S}_u, \mathcal{E}_u, q_u)$$

When $|\mathcal{C}|$ exceeds the batch size, candidates are partitioned into batches scored independently. The final ranking is obtained by sorting candidates in descending order of r_j .

4 Experiments

4.1 Experiment Setup

Datasets. We evaluate on four domains from the InstructRec benchmark (Xu et al., 2025), which augments Amazon product reviews (Ni et al., 2019) (**Books**, **MovieTV**), Goodreads (Wan et al., 2019) (**Goodreads**), and **Yelp** (Yelp, 2024) with natural language instructions. The four domains span a range of sparsity levels and interaction patterns, as summarized in Table 1. Retrospective experiments are conducted on all four domains with full user sets, while evolving experiments use two Books subsamples (100 users each) due to the high per-signal LLM computational cost: *Active User* and *Light User*, allowing us to study how evolving memory updates perform across users with rich versus limited history.

Table 1 Dataset statistics. Density = Interactions / (Users × Items).

Dataset	Users	Items	Interactions	Density	Avg/User
Books	7,377	120,925	207,759	0.023%	28.2
Goodreads	11,734	57,364	618,330	0.092%	52.7
MovieTV	5,649	28,987	79,737	0.049%	14.1
Yelp	2,950	31,636	63,142	0.068%	21.4
Books (Active User)	100	8,334	8,935	1.072%	89.3
Books (Light User)	100	1,505	1,534	1.019%	15.3

Evaluation Modes. To comprehensively validate the proposed approach, we evaluate under two complementary settings. *Retrospective* mode follows the standard protocol used by prior work (Chen et al., 2026; Xu et al., 2025): the full interaction history $\mathcal{H}_u$ is processed at a single point in time to build the memory state M_u . Concretely, the extraction step ingests all interactions to construct preference memory $\mathcal{P}_u$ and synthesize profile memory $\mathcal{S}_u$, while event memory $\mathcal{E}_u$ retains only the most recent L_{max} signals as ranking context. This mode enables direct comparison with baselines under identical conditions. *Evolving* mode further validates the memory lifecycle by simulating online operation, where memory evolves continuously as behavioral signals arrive sequentially. We evaluate two scheduling strategies: *fixed*, which triggers memory operations every B signals, and *agentic*, where π_{plan} adaptively decides when updates are warranted. Unless otherwise stated, we report results in retrospective mode.

Evaluation Metrics. Following prior work (Chen et al., 2026; Xu et al., 2025), we use a leave-one-out evaluation protocol where the most recent interaction per user is held out for testing. Each test instance consists of 1 ground-truth item and 9 randomly sampled negative items ($N = 10$ candidates). We report HR@1, HR@5, NDCG@5, and NDCG@10.

Table 2 Retrospective mode results across four domains. Best in **bold**, second best underlined. H, N denote HR, NDCG. Improv. shows % gain of MARS over second best.

Model	Books				Goodreads				MovieTV				Yelp			
	H@1	H@5	N@5	N@10	H@1	H@5	N@5	N@10	H@1	H@5	N@5	N@10	H@1	H@5	N@5	N@10
BPR	.140	.563	.340	.483	<u>.341</u>	.722	<u>.539</u>	<u>.627</u>	.263	.622	.441	.562	.231	.666	.444	.553
LightGCN	.152	.574	.343	.486	.285	.640	.466	.580	.330	.671	.499	.607	.313	<u>.817</u>	.562	.624
SASRec	.198	.707	.464	.559	.234	.685	.462	.563	.188	.483	.337	.500	.198	.454	.332	.497
LLMRank	.215	.602	.416	.539	.218	.603	.417	.540	.294	.716	.513	.601	.265	.520	.398	.546
AgentCF	.263	.620	.445	.562	.124	.380	.251	.438	.171	.365	.271	.458	.150	.479	.315	.474
iAgent	.335	.664	.499	.605	.162	.593	.377	.505	.380	.749	.572	.650	.289	.681	.481	.582
i2Agent	.325	.716	.526	.613	.164	.662	.417	.522	.349	.768	.569	.640	.259	.673	.468	.569
MemRec	<u>.396</u>	<u>.770</u>	<u>.591</u>	<u>.662</u>	.257	<u>.741</u>	.500	.584	<u>.471</u>	<u>.850</u>	<u>.668</u>	<u>.715</u>	<u>.402</u>	.757	<u>.590</u>	<u>.665</u>
MARS	.499	.846	.682	.731	.369	.770	.568	.643	.591	.896	.752	.785	.587	.887	.752	.787
Improv.	+26.0	+9.9	+15.4	+10.4	+8.2	+3.9	+5.4	+2.6	+25.5	+5.4	+12.6	+9.8	+46.0	+8.6	+27.5	+18.3

Baselines. We compare against both traditional recommendation models and LLM-based recommendation agents:

- **BPR** (Rendle et al., 2009): Bayesian personalized ranking with matrix factorization, optimizing pairwise item preferences.
- **LightGCN** (He et al., 2020): Graph collaborative filtering with linear embedding propagation over the user-item interaction graph.
- **SASRec** (Kang and McAuley, 2018): Self-attentive sequential model that captures long-term dependencies in user interaction sequences.
- **LLMRank** (Hou et al., 2024): Zero-shot LLM ranking without persistent user memory.
- **AgentCF** (Zhang et al., 2024b): User and item agents with collaborative memory updated through pairwise interaction and reflection.
- **iAgent / i2Agent** (Xu et al., 2025): User-agent paradigm with static (iAgent) and iterative (i2Agent) profile generation via feedback loops.
- **MemRec** (Chen et al., 2026): Decoupled memory manager and reasoning LLM with collaborative memory graph propagation.

Implementation Details. Traditional baselines are trained using RecBole (Zhao et al., 2021). All LLM-based methods use Llama-4-Maverick-17B-128E-Instruct (Meta AI, 2025) as the default backbone LLM unless otherwise specified (see LLM Backbone ablation in §4.4). Following i2Agent (Xu et al., 2025), all memory-based approaches use a history window of $L_{\max} = 15$ recent interactions (full hyperparameters in Appendix B).

4.2 Retrospective Mode Results

Table 2 presents the retrospective results across all four InstructRec domains.

MARS achieves the best performance on all four metrics across all four domains, as shown in the Improv. rows. HR@1 gains range from 8.2% (Goodreads) to 46.0% (Yelp), with improvements consistently larger on HR@1 than on deeper-rank metrics, indicating that the three-tier memory architecture helps the ranker precisely identify the single most relevant item by providing fine-grained preference resolution.

Among LLM-based methods, MemRec is the strongest competitor, ranking second on Books and MovieTV across most metrics. Notably, traditional CF baselines remain competitive in denser domains: BPR is second-best on three of four metrics on Goodreads, the densest dataset (0.092% density), and LightGCN achieves second-best HR@5 on Yelp, suggesting that dense collaborative signals partially compensate for the absence of semantic reasoning. Nevertheless, MARS consistently outperforms both categories by combining structured memory with language-based ranking.

Table 3 Evolving mode results on Books. Best in **bold**, second best underlined.

Model	Active User				Light User			
	H@1	H@5	N@5	N@10	H@1	H@5	N@5	N@10
AgentCF	.230	.650	.443	.554	.340	.640	.488	.604
i2Agent	.490	.750	<u>.635</u>	<u>.717</u>	<u>.600</u>	.850	<u>.729</u>	<u>.776</u>
MemRec	.360	.730	.554	.638	.450	.800	.636	.697
MARS (fixed)	<u>.500</u>	<u>.760</u>	.632	.710	.510	<u>.860</u>	.692	.736
MARS (agentic)	.510	.830	.678	.733	.620	.930	.784	.807
Improv.	+2.0	+9.2	+6.8	+2.2	+3.3	+8.1	+7.5	+4.0

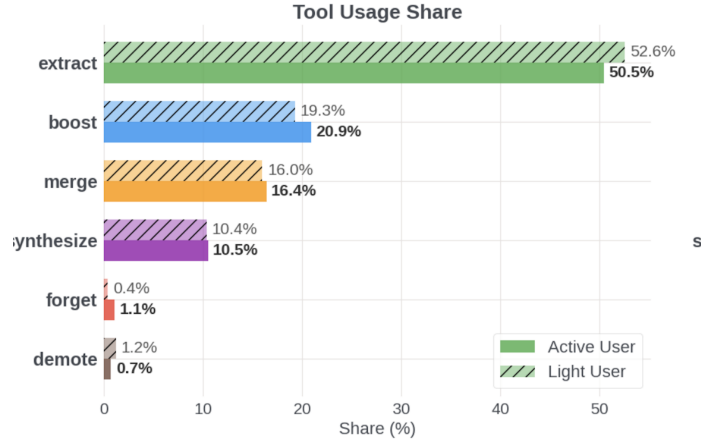


Figure 2 Planner tool usage distribution on Books: extract and boost dominate ($\sim 70\%$), while forget and demote account for $< 2\%$.

4.3 Evolving Mode Results

To validate the memory lifecycle under online operation, we evaluate on the Books Active User and Light User subsamples in evolving mode. We compare against AgentCF, MemRec, and i2Agent, the approaches that support or can be adapted to evolving memory maintenance. Table 3 presents results for these baselines and two MARS scheduling strategies: *fixed*, which triggers memory operations every B signals, and *agentic*, where π_{plan} adaptively decides when updates are warranted. Both MARS variants incorporate user instructions at ranking time when available.

MARS (agentic) achieves the best results across all metrics on both user groups, while MARS (fixed) remains competitive, surpassing all baselines on Active Users and matching i2Agent on Light Users. The agentic advantage over fixed scheduling is substantially larger on Light Users (+21.6% HR@1) than Active Users (+2.0%), because limited history makes each scheduling decision more impactful. A detailed case study tracing how the planner uses demote and forget to handle a preference shift for a single user is provided in Appendix F.

4.4 Ablation Studies

Memory Architecture Tiers. We evaluate the contribution of each memory tier through a full factorial study ($2^3 = 8$ combinations of $\pm$ event, $\pm$ preference, $\pm$ profile memory) on the Books dataset with all 7,377 users (Figure 3). Two observations emerge. **(1) Any memory dramatically outperforms none, and profile+event achieves the best results.** The no-memory baseline (NDCG@10 = 0.424) underperforms every single-tier configuration by $> 60\%$ relative, confirming that persistent memory is essential. The profile+event combination achieves the highest NDCG@10 (0.731), validating our design choice of feeding only profile and event memory to the ranker while using preference memory as an intermediate lifecycle tier. **(2) Preference memory contributes through profile synthesis, not direct ranking input.** Adding preference chunks directly to the ranking prompt alongside profile and event memory (all three tiers: 0.725) slightly underperforms profile+event alone (0.731). This

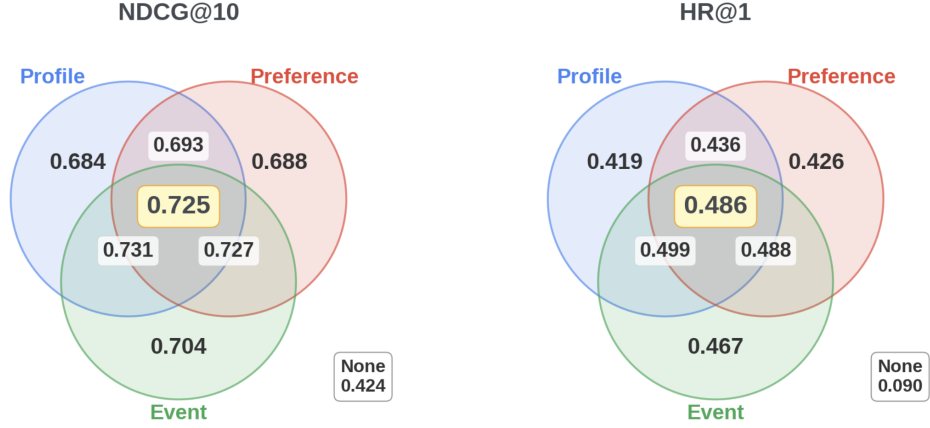


Figure 3 Memory tier ablation on Books (7,377 users). Left: NDCG@10; right: HR@1. Profile+event (0.731) achieves the best NDCG@10.

Table 4 Planner LLM ablation in evolving agentic mode on Books. Ranking and memory fixed to Maverick-17B; only the planner LLM varies. Best in **bold** within each sample group.

Planner	Active User				Light User			
	H@1	H@5	N@5	N@10	H@1	H@5	N@5	N@10
Llama-4-17B	.510	.830	.678	.733	.620	.930	.784	.807
Llama-3-8B	.490	.790	.651	.719	.650	.930	.793	.815
Gemini-3-Flash	.540	.840	.698	.750	.680	.950	.814	.830

confirms that preference memory’s value lies in structuring the lifecycle — enabling fine-grained extraction, boosting, and forgetting that produce a higher-quality profile — rather than as a direct signal for ranking, where it introduces redundancy with the profile that already summarizes it.

LLM Backbone. We study the effect of varying the planner LLM in evolving agentic mode while fixing ranking and memory to Maverick-17B (Table 4). We compare three models spanning different scales: Llama-4-Maverick-17B-128E-Instruct (Meta AI, 2025) (the default), Llama-3.1-8B-Instruct (a smaller, faster alternative), and Gemini-3-Flash (Google DeepMind, 2026).

Gemini-3-Flash achieves the best results as a planner on both Active Users (HR@1=0.540, +5.9% over Maverick) and Light Users (HR@1=0.680, +9.7%), despite being a smaller model. This indicates that planning and ranking require different LLM capabilities: planning benefits from fast, decisive instruction-following, while ranking requires deeper semantic reasoning over item descriptions and user preferences. Llama-3.1-8B as planner also surpasses Maverick on Light Users (0.650 vs. 0.620), suggesting that a smaller, faster planner can be equally or more effective, with practical computational cost implications for deployment.

4.5 Computational Cost Analysis

We analyze computational efficiency by measuring per-user LLM token consumption across all approaches on Books using Llama-4-Maverick-17B. Figure 4 shows the computational cost–quality tradeoff: stacked bars decompose total tokens into input (prompt) and output (completion), while the overlaid line tracks recommendation quality.

MARS uses only 3 LLM calls per user totaling 3,870 tokens, compared to MemRec’s 6 calls and 8,849 tokens—a 2.3× reduction achieved by compressing interaction history into compact preference chunks and replacing multi-stage pipelines with a single ranking call.

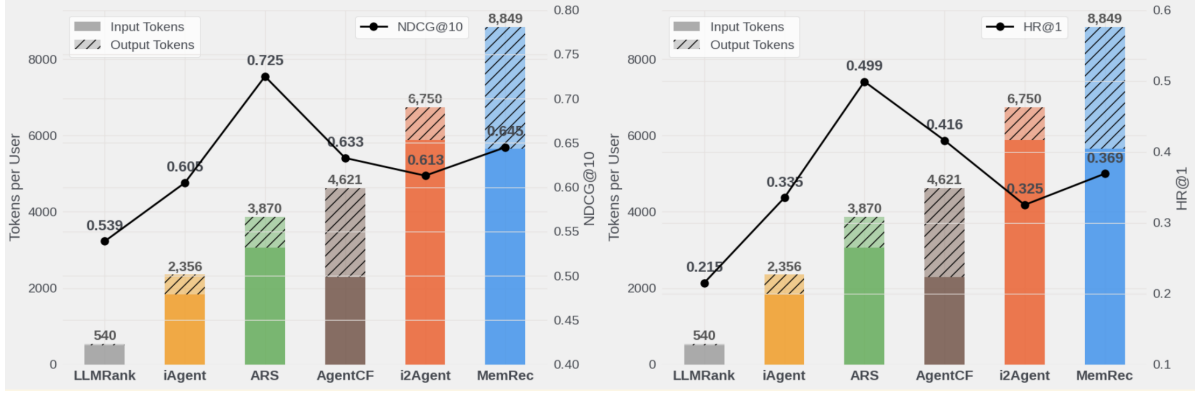


Figure 4 Computational cost-quality tradeoff (Books, Maverick-17B). Bars: per-user tokens (input + output). Line: recommendation quality.

5 Conclusion

We presented MARS, an agentic recommendation framework that maintains a three-tier belief state (event, preference, profile) through an LLM-governed lifecycle of six operations. Experiments across four domains show 26.4% average HR@1 and 10.3% NDCG@10 gains over the strongest baselines at $2.3\times$ fewer tokens, with agentic scheduling yielding up to 21.6% further gains in evolving settings. The ablation reveals that preference memory’s value lies not in directly informing the ranker, but in structuring the lifecycle operations that produce a higher-quality profile — acting as the intermediate state representation through which noisy observations are refined into a compact belief state. This suggests that in partially observable preference modeling, the quality of the state abstraction pipeline matters more than the richness of the state fed to the decision-maker.

References

- Nicolas Bougie and Narimasa Watanabe. Simuser: Simulating user behavior with large language models for recommender system evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, 2025.
- Micah Carroll, Adeline Foote, Kevin Feng, Marcus Williams, Anca Dragan, W. Bradley Knox, and Smitha Milli. Ctrl-rec: Controlling recommender systems with natural language. *arXiv preprint arXiv:2510.12742*, 2025.
- Luyu Chen, Quanyu Dai, Zeyu Zhang, Xueyang Feng, Mingyu Zhang, Pengcheng Tang, Xu Chen, Yue Zhu, and Zhenhua Dong. Recusersim: A realistic and diverse user simulator for evaluating conversational recommender systems. In *Proceedings of the ACM Web Conference 2025, Industry Track*, 2025.
- Weixin Chen, Yuhan Zhao, Jingyuan Huang, Zihe Ye, Clark Mingxuan Ju, Tong Zhao, Neil Shah, Li Chen, and Yongfeng Zhang. Memrec: Collaborative memory-augmented agentic recommender system. *arXiv preprint arXiv:2601.08816*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Yu Deng, Jianxun Lian, Yuxuan Lei, Chongming Gao, Kexin Huang, and Jiawei Chen. Recbot: Agent-based recommendation system. *arXiv preprint arXiv:2509.21317*, 2025.
- Google DeepMind. Gemini 3. <https://deepmind.google/models/gemini/>, 2026. Gemini 3 Flash. Accessed April 2026.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.
- Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *Proceedings of the 46th European Conference on Information Retrieval (ECIR)*, 2024.
- Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. Recommender ai agent: Integrating large language models for interactive recommendations. *arXiv preprint arXiv:2308.16505*, 2023.
- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99-134, 1998.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. Oral presentation.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 2018.
- Zhefan Lei, Hengxu Wang, Jiawei Zhang, and Shuai Chen. Macrec: A multi-agent collaboration framework for recommendation. *arXiv preprint arXiv:2402.15235*, 2024.
- Bingqian Li, Xiaolei Wang, Junyi Li, Weitao Li, Long Zhang, Sheng Chen, Wayne Xin Zhao, and Ji-Rong Wen. Recnet: Self-evolving preference propagation for agentic recommender systems. *arXiv preprint arXiv:2601.21609*, 2026.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.
- Yuxin Liao, Le Wu, Min Hou, Yu Wang, Han Wu, and Meng Wang. From atom to community: Structured and evolving agent memory for user behavior modeling. *arXiv preprint arXiv:2601.16872*, 2026.
- Zhongqi Lu and Qiang Yang. Partially observable markov decision process for recommender systems. *arXiv preprint arXiv:1608.07793*, 2016.
- Hanjia Lyu, Song Jiang, Hanqing Zeng, Yinglong Xia, Qifan Wang, Si Zhang, Ren Chen, Christopher Leung, Jiajie Tang, and Jiebo Luo. Llm-rec: Personalized recommendation via prompting large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.
- Meta AI. Llama 4. https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md, April 2025. Llama 4 Scout (17B, 16 experts) and Llama 4 Maverick (17B, 128 experts). Natively multimodal mixture-of-experts models.

- Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Sungjoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091*, 2019.
- Minh-Duc Nguyen, Hai-Dang Kieu, and Dung D. Le. Amem4rec: Leveraging cross-user similarity for memory evolution in agentic llm recommenders. *arXiv preprint arXiv:2602.08837*, 2026.
- Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 188–197, 2019.
- Kesha Ou, Chenghao Wu, Xiaolei Wang, Bowen Zheng, Wayne Xin Zhao, Weitao Li, Long Zhang, Sheng Chen, and Ji-Rong Wen. Deep research for recommender systems. *arXiv preprint arXiv:2603.07605*, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Qiyao Peng, Hongtao Liu, Hua Huang, Qing Yang, and Minglai Shao. A survey on llm-powered agents for recommender systems. *arXiv preprint arXiv:2502.10050*, 2025.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (UAI)*, 2009.
- Yunxiao Shi, Wujiang Xu, Zeqi Zhang, Xing Zi, Qiang Wu, and Min Xu. Personax: A recommendation agent oriented user modeling framework for long behavior sequence. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- Theodore Sumers, Shunyu Yao, Karthik R Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.
- Mengting Wan, Rishabh Misra, Ndapa Nakashole, and Julian McAuley. Fine-grained spoiler detection from large-scale review corpora. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2605–2610, 2019.
- Lei Wang, Jingsen Zhang, Hao Yang, Zhiyuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Ruihua Song, Wayne Xin Zhao, Jun Xu, Zhicheng Dou, Jun Wang, and Ji-Rong Wen. User behavior simulation with large language model based agents. *arXiv preprint arXiv:2306.02552*, 2023.
- Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Yanbin Lu, Xiaojiang Huang, and Yingbo Lu. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296*, 2024.
- Wujiang Xu, Yunxiao Shi, Zujie Liang, Xuying Ning, Kai Mei, Kun Wang, Xi Zhu, Min Xu, and Yongfeng Zhang. iagent: Llm agent as a shield between user and recommender systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- Zihe Ye, Jingyuan Huang, Weixin Chen, and Yongfeng Zhang. H-mem: Hybrid multi-dimensional memory management for long-context conversational agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7756–7775, 2026.
- Yelp. Yelp open dataset. <https://www.yelp.com/dataset>, 2024.
- Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- An Zhang, Yuxin Chen, Leheng Sheng, Xiang Wang, and Tat-Seng Chua. On generative agents in recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024a.
- Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001*, 2023.

- Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM Web Conference (WWW)*, 2024b.
- Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, et al. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, 2021.
- Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten de Rijke. Let me do it for you: Towards llm empowered recommendation via tool learning. *arXiv preprint arXiv:2405.15114*, 2024.
- Hailin Zhong, Hanlin Wang, Yujun Ye, Meiyi Zhang, and Shengxin Zhu. Ggbond: Growing graph-based ai-agent society for socially-aware recommender simulation. *arXiv preprint arXiv:2505.21154*, 2025.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

A Agentic Memory Update Algorithm

Algorithm 1 Agentic Memory Update

Require: User u , interaction stream $\mathcal{H}_u$, memory state $M_u = (\mathcal{E}_u, \mathcal{P}_u, \mathcal{S}_u)$

Require: Planner check interval B , synthesis threshold γ

```

1: mutations  $\leftarrow 0$ 
2: for each new behavioral signal  $e$  do
3:    $\mathcal{E}_u \leftarrow \mathcal{E}_u \cup \{e\}$  ▷ Append to event memory
4:   if  $|\mathcal{E}_u^{\text{pending}}| \geq B$  then
5:      $a_t \leftarrow \pi_{\text{plan}}(\mathcal{S}_u, \mathcal{P}_u, \mathcal{E}_u^{\text{pending}}, H_{\text{prev}})$  ▷ Schedule ops
6:     for each  $(t_k, \text{params}_k) \in a_t$  do
7:       Execute operation  $t_k$  on  $M_u$ 
8:       mutations  $\leftarrow$  mutations + 1
9:     end for
10:    Apply per-category capacity constraint to  $\mathcal{P}_u$ 
11:    if mutations  $\geq \gamma$  then
12:       $\mathcal{S}_u \leftarrow \pi_{\text{syn}}(\mathcal{P}_u, \mathcal{S}_u)$  ▷ Auto-synthesize
13:      mutations  $\leftarrow 0$ 
14:    end if
15:  end if
16: end for

```

B Supplemental Implementation Details

MARS Hyperparameters. Table 5 summarizes the full set of hyperparameters used for MARS.

Table 5 MARS hyperparameter settings.

Parameter	Symbol	Value
Planner check interval	B	3
Max preferences per category	K_{cat}	8
Event memory window	L_{max}	15
Boost step	δ_+	0.1
Demote step	δ_-	0.2
Forgetting strength threshold	ϵ	0.1
Forgetting evidence threshold	n_{min}	5
Synthesis trigger	γ	5 mutations
Candidate slate size	N	10
Concurrent evaluation batch	—	32 users

Decoding Configuration. All LLM calls across all methods use temperature 0 (greedy decoding) to ensure deterministic outputs and minimize stochasticity across runs. No sampling, top- k , or nucleus filtering is applied.

User Subsample Selection. Evolving mode experiments use two 100-user subsamples from the Books domain, selected by interaction count and a fixed random seed for reproducibility. *Active Users*: users with 50–200 interactions, sampled with seed 42. *Light Users*: users with 10–30 interactions, sampled with seed 64. The same user sets are used consistently across all evolving experiments and ablations to ensure comparability.

Preference Categories. Each domain uses six preference categories that structure the preference memory tier. Categories are generated once per domain by prompting the LLM with 10 sample item descriptions and asking it to identify the most discriminative axes of preference. Table 6 lists the resulting categories.

Table 6 Preference categories per domain, generated by a one-time LLM call on 10 sample item descriptions.

Domain	Categories
Books	genre, writing style, theme, setting, author type, mood
Goodreads	genre, writing style, theme, series pref., mood, character type
MovieTV	genre, mood, era, theme, quality, pacing
Yelp	cuisine, price range, ambiance, occasion, location, service

C Additional Ablation Studies

Memory Capacity. Table 7 shows the effect of varying the maximum preferences per category K_{cat} on Books (all users). Performance is stable across $K_{\text{cat}} \in \{2, 4, 8, 16\}$, with HR@1 ranging from 0.486 to 0.499 and NDCG@10 from 0.725 to 0.731. This robustness suggests that the consolidation and forgetting mechanisms effectively manage preference quality regardless of capacity, and that a small number of high-confidence preferences per category suffices for ranking. In summary, preference capacity is not a sensitive hyperparameter for MARS, and practitioners can set K_{cat} without careful tuning.

Table 7 Effect of preference capacity K_{cat} on Books (all users). Performance is stable across all settings.

K_{cat}	H@1	H@5	N@5	N@10
2	.490	.844	.679	.728
4	.486	.845	.676	.725
8 (default)	.499	.846	.682	.731
16	.487	.845	.677	.726

D Computational Cost Estimation

Table 8 estimates the total LLM token consumption and USD computational cost for MARS across all evaluation settings, using Gemini 2.5 Flash pricing (\$0.30/M input, \$2.50/M output) as a reference.

Table 8 Estimated MARS token consumption and USD computational cost. USD computed using Gemini 2.5 Flash pricing (\$0.30/M input, \$2.50/M output); costs scale linearly with model pricing tier.

Mode	Domain	Users	Tokens/User	Total (M)	Est. USD
Retrospective	Books	7,377	3,870	28.5	\$21.75
Retrospective	Goodreads	11,734	3,870	45.4	\$34.74
Retrospective	MovieTV	5,649	3,870	21.9	\$16.69
Retrospective	Yelp	2,950	3,870	11.4	\$8.70
Evolving	Active User	100	~68,500	6.8	\$5.12
Evolving	Light User	100	~12,800	1.3	\$1.05
Total		27,910	–	115.3	\$88.05

At Gemini 2.5 Flash pricing (\$0.30/M input tokens, \$2.50/M output tokens), the total computational cost for all MARS experiments is approximately \$88. Traditional baselines (BPR, LightGCN, SASRec) are trained with RecBole on a single GPU with negligible computational cost relative to LLM inference.

E Limitations

Scope of Evaluation. This work focuses on LLM-native recommendation with natural language instructions available, a setting where structured memory and language-based reasoning provide the greatest advantage. In this setting, MARS consistently outperforms both traditional collaborative filtering models and LLM-based baselines. However, on dense domains where collaborative signals are strong (e.g., Goodreads), traditional models such as BPR remain competitive, suggesting that the current framework is most beneficial when semantic reasoning over instructions and item descriptions plays a central role.

Evolving Evaluation Scale and Variance. Evolving mode experiments use 100-user subsamples due to the high per-signal LLM computational cost of sequential memory updates. This scale is consistent with prior work in the same setting (Zhang et al., 2024b; Nguyen et al., 2026), which evaluate on comparable or smaller user populations. We do not report error bars; however, all LLM calls use temperature 0 (greedy decoding) to minimize stochasticity. Residual variance arises only from non-deterministic infrastructure factors such as GPU kernel scheduling, floating-point accumulation order, and batched request routing, which are negligible in practice. The memory capacity ablation (Table 7) corroborates this: performance varies by at most 0.013 in HR@1 and 0.006 in NDCG@10 across four hyperparameter settings, indicating high stability.

Absence of Collaborative Signals. MARS operates on a per-user basis and does not propagate preference updates across users. While collaborative signals have been shown to benefit recommendation in prior work (Chen et al., 2026; Nguyen et al., 2026; Li et al., 2026), the three-tier architecture is designed to be modular: collaborative memory components (e.g., shared preference graphs, cross-user memory propagation) can be integrated as additional tools available to the planner without modifying the core lifecycle. Exploring this integration is an important direction for future work.

F Case Study: Memory Lifecycle in Action

We illustrate the memory lifecycle on a representative user from the Books evolving evaluation (User 1773, 65 training interactions, HR@1 = 1.0). This user is a theology reader whose interests briefly shift toward historical biographies before returning to their core domain. The agentic planner uses five of six lifecycle operations—extract, boost, demote, forget, and synthesize—to track this preference evolution.

Phase 1: Discovery (interactions 1–30). The planner extracts core preferences: topics such as “Christianity” (0.8), “spiritual formation” (0.7), “reformed theology” (0.8), and authors including “Dietrich Bonhoeffer” (0.9) and “Michael Horton” (0.8). By step 15, the user has 11 preference chunks across two categories.

Phase 2: Preference Shift (interactions 30–40). The user reads books by Churchill and Lincoln, which the planner extracts as new author preferences (strength 0.7 each). However, subsequent interactions return to theology. The planner recognizes these as tangential and issues two rounds of **demote** operations:

- Round 1: demote “Churchill” (0.7 $\rightarrow$ 0.5) and “Lincoln” (0.7 $\rightarrow$ 0.5), while simultaneously extracting “postmodern christianity” and “Henri Nouwen”.
- Round 2: demote “Churchill” (0.5 $\rightarrow$ 0.3) and “Lincoln” (0.5 $\rightarrow$ 0.3), while extracting “immigration” and “Douglas Murray”.

The preference count dips from 15 to 14 at this stage, reflecting the planner’s active curation.

Phase 3: Pruning and Reinforcement (interactions 40–50). The demoted preferences are now weak enough to trigger **forget**: the planner removes “Churchill” and “Lincoln” entirely, while simultaneously **boosting** “reformed theology” (0.8 $\rightarrow$ 0.9 $\rightarrow$ 1.0) and “John Calvin” (0.8 $\rightarrow$ 0.9 $\rightarrow$ 1.0). Profile **resynthesis** fires automatically after every 5 mutations, producing updated narratives that reflect the refined preference state.

Phase 4: Continued Exploration (interactions 50–65). The planner continues extracting niche theology topics (“covenant theology”, “lord’s supper”, “beatitudes”, “Heidelberg Catechism”) and periodically boosting “reformed theology”. The preference count stabilizes at 16.

Result. The final memory state contains 16 preferences across 2 categories (topic, author), with 5 profile resyntheses performed over the session. The ranker, consuming the synthesized profile and recent event signals, ranks the ground-truth test item first (HR@1 = 1.0). The lifecycle tool distribution for this user (extract 52%, boost 23%, synthesize 11%, demote 9%, forget 5%) illustrates all three phases of memory management: discovery, belief revision, and consolidation.

G Prompt Templates

This section presents the core LLM prompt templates used in MARS. Variable placeholders are shown in {braces}.

Prompt 1: Preference Extraction (π_{ext})

System: You are analyzing user engagement to understand their preferences.

Given these recent engagements with {item_noun}:
{engagements_table}

Current known preferences:
{existing_preferences}

Extract or update preference statements. For each preference:

1. **category:** {category_list}
2. **text:** Natural language statement (be specific, not generic)
3. **strength:** -1.0 (strong dislike) to 1.0 (strong like)
4. **action:** “create” | “strengthen” | “weaken”

Rules:

- Be specific: “Enjoys cerebral sci-fi with philosophical themes” > “Likes sci-fi”
- Look for patterns: 3 similar engagements → preference, 1 → too early
- Maximum 5 preferences per response

Output as JSON array only, no other text.

Prompt 2: Profile Synthesis (π_{syn})

System: You are a user preference analyst. Write concise, coherent profiles.

Group these preferences into a coherent 150–300 word profile paragraph that captures this user’s taste. Write in third person (“This user...”).

Preference statements:
{chunks_text}

Previous profile (maintain continuity):
{previous_profile}

Write a single coherent paragraph (150–300 words). Do NOT use bullet points. Output ONLY the profile paragraph.

Prompt 3: Agentic Scheduler (π_{plan})

System: You are a memory manager for a recommendation system. Analyze preferences against new items. Output ONLY valid JSON.

User Profile: {full_profile}

Current Preferences ({pref_count} chunks): {preference_list}

New Items ({pending_count} pending): {items_with_descriptions}

Memory Health: {health_section}

Available Actions:

1. **extract** – Create new preferences
2. **merge** – Merge similar preferences
3. **boost** – Reinforce a preference
4. **demote** – Weaken a preference
5. **forget** – Delete a preference

Guidelines:

- **Skip is the default.** If new items match existing preferences, output {“actions”: []}
- Only extract when items reveal genuinely NEW interests
- Be selective: at most 3 actions per response

Output: {“actions”: [...]} or {“actions”: []}

Prompt 4: LLM Ranking (π_{rank})

You are a recommendation engine. Score how well each item matches this user's preferences.

{user_profile_section}
{session_memory_section}
{instruction_section}

Items to Score:
{items_table}

Instructions: Score each item on a 0–10 scale:
– 8–10: *STRONG* match – high confidence user will enjoy
– 4–7: *MAYBE* – moderate match
– 0–3: *WEAK* – poor match

Use the full 0–10 range. Spread scores apart. Best match should score 9+, worst should score 2 or below.

Output Format (one line per item, no extra text):
ITEM_ID | SCORE | TIER | brief_reason